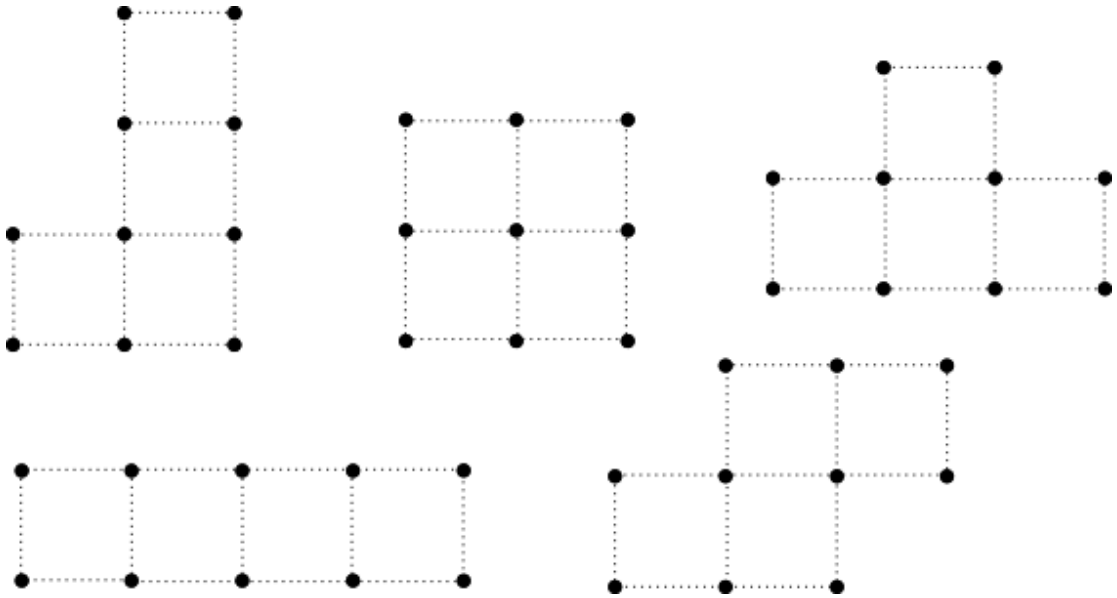


Identifying tetris blocks with a graph convolutional neural network

In this exercise, we will build and train a graph neural network to identify tetris blocks from the graph formed by their mesh:



```
In [1]: from typing import List, Callable, Tuple

import torch
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt
```

Creating the dataset

First, we define the 2d positions of the nodes of the mesh. In the GNN, this information will not be available, but only the connections between the nodes.

```
In [2]: lblock = torch.tensor([[0.,0.], [1.,0.], [2.,0.], [0.,1.], [1.,1.], [2.,1]
square = torch.tensor([[0.,0.], [1.,0.], [2.,0.], [0.,1.], [1.,1.], [2.,1]
line = torch.tensor([[0.,0.], [1.,0.], [2.,0.], [3.,0.], [4.,0.], [0.,1.]
step = torch.tensor([[0.,0.], [1.,0.], [2.,0.], [0.,1.], [1.,1.], [2.,1.]
tblock = torch.tensor([[0.,0.], [1.,0.], [2.,0.], [3.,0.], [0.,1.], [1.,1]
tetris_blocks = [lblock, square, line, step, tblock]
```

Next, we need to compute the edges of the mesh. Every pair of points that has a distance less or equal to a given cutoff (default: 1) shall be connected.

Task: Complete the function `compute_edges` so that it returns a list of edges for the passed tensor of points. You can check your solution in the cell below, which creates the dataset and plots the tetris blocks. Note, that according to

this definition self-connections \$(i,i)\$ should be included.

```
In [3]: def compute_edges(points:torch.FloatTensor, cutoff:float=1.) -> torch.LongTensor:
        """
        This function computes the edges of graph given an array of nodes with shape (N, 2).
        The graph will be created by connecting all nodes with an euclidean distance less than
        the provided cutoff.

        Args:
        - points: tensor of coordinates with shape (num_nodes x 2)
        - cutoff: cutoff distance to determine whether two points are connected

        Returns:
        - edges: tensor of shape [num_edges x 2] where each row corresponds to a pair of
          connected nodes (incoming, outgoing)

        """
        N, _ = points.shape
        points_f = points.to(torch.float64)
        D = torch.cdist(points_f, points_f)
        x, y = torch.meshgrid(torch.arange(0, N), torch.arange(0, N))
        edges = torch.stack((x[D <= cutoff], y[D <= cutoff])).T
        return edges.view(-1, 2)
```

```
In [4]: def create_dataset(shuffle:bool=False):
        tetris_edges = []
        tetris_points = []
        for points in tetris_blocks:
            if shuffle:
                sort_idx = torch.randperm(points.shape[0])
                points = points[sort_idx]
                tetris_points.append(points)

            edges = compute_edges(points)
            tetris_edges.append(edges)
        return tetris_points, tetris_edges

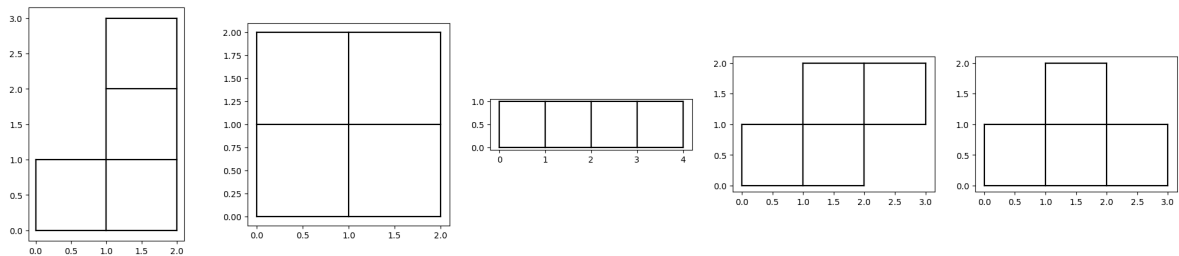
tetris_points, tetris_edges = create_dataset(shuffle=False)

plt.figure(figsize=(25, 5))
for i, graph in enumerate(zip(tetris_points, tetris_edges)):
    block, edges = graph
    plt.subplot(1, 5, i+1)
    X = torch.vstack([block[edges[:,0],0], block[edges[:,1], 0]])
    Y = torch.vstack([block[edges[:,0],1], block[edges[:,1], 1]])
    plt.plot(X, Y, c='k')
    plt.gca().set_aspect('equal')

plt.show()
```

```
/nix/store/35mnrms5q07fzfbjyppy8ypc2752g00f-python3-3.13.12-env/lib/python
3.13/site-packages/torch/functional.py:505: UserWarning: torch.meshgrid: in
an upcoming release, it will be required to pass the indexing argument.
(triggered internally at /build/pytorch/aten/src/ATen/native/TensorShape.c
pp:4381.)
```

```
return _VF.meshgrid(tensors, **kwargs) # type: ignore[attr-defined]
```



Finally, a batched version of the dataset is created. Since the graphs have different numbers of nodes and edges, we can not use a separate batch dimension. Instead, we need to use a global, sparse representation where nodes of different graphs have unique indices.

We describe the graph with the following tensors:

- `batched_node_labels` : The node labels are the main input to the graph network and have a shape `(num_nodes, 1)`. Since we don't have different types of nodes, we just label every node 1 .
- `batched_edges` : The edges, you have computed in your first task, are stacked and indices are converted to be compatible with the `batched_node_labels` . E.g. for two graphs with 3 and 5 nodes, respectively, the node indices will be 0-2 for the first and 3-7 for the second graph.
- `graph_idx` : Since we do not have an explicit batch dimension, we need to store here to which graph a node belongs.
- `classes` : The labels for classification. Since we aim to identify each tetris block, we use here the indices 0-4.

```
In [5]: def build_batch(edges_list:List[torch.Tensor]) -> Tuple[torch.Tensor, torch.Tensor, torch.Tensor, torch.Tensor]:
    num_nodes = torch.tensor([torch.max(edges)+1 for edges in edges_list])
    classes = torch.arange(len(tetris_edges), dtype=torch.long)
    block_idx = torch.repeat_interleave(classes, num_nodes)

    # convert node indices to global batch indices
    node_offsets = torch.cumsum(torch.cat([torch.zeros((1,)), dtype=torch.long]), 0)
    global_edges_list = [edges + node_offsets[i] for i, edges in enumerate(edges_list)]
    batched_node_labels = torch.ones((int(torch.sum(num_nodes).item()), 1), dtype=torch.long)
    batched_edges = torch.vstack(global_edges_list)
    return batched_node_labels, batched_edges, block_idx, classes

batch = build_batch(tetris_edges)

print(f'batched_node_labels [first 10 entries]: {batch[0][:10]}')
print(f'batched_edges [first 10 entries]: {batch[1][:10]}')
print(f'block_idx: {batch[2]}')
print(f'classes: {batch[3]}')
```

```

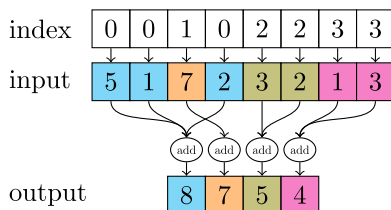
batched_node_labels [first 10 entrie]: tensor([[1.],
        [1.],
        [1.],
        [1.],
        [1.],
        [1.],
        [1.],
        [1.],
        [1.],
        [1.]])
batched_edges [first 10 entrie]: tensor([[0, 0],
        [0, 1],
        [0, 3],
        [1, 0],
        [1, 1],
        [1, 2],
        [1, 4],
        [2, 1],
        [2, 2],
        [2, 5]])
block_idx: tensor([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        2, 2, 2, 2, 2,
        2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 4, 4, 4, 4, 4, 4, 4,
        4, 4,
        4])
classes: tensor([0, 1, 2, 3, 4])

```

Graph Convolutional Neural Network

In this section we will implement the GNN and its required layers and ops.

First, we will implement a function that sums over nodes and edges of a graph. The operation we need is called `scatter_add`, which takes an input tensor `x` and an index tensor `idx` :
$$\text{out}_i = \sum_j x_j \mathbb{1}_{\{\text{idx}_j=i\}}$$
 `Scatter_add` is visualized by the following picture:



Task: Implement `scatter_add` below. You can use the PyTorch library function `index_add` for that.

```

In [6]: def scatter_add(
        x: torch.Tensor, idx: torch.Tensor, dim_size: int, dim: int = 0
    ) -> torch.Tensor:
        res_shape = list(x.shape)
        res_shape[dim] = dim_size
        res = torch.zeros(res_shape, dtype=x.dtype)

        slices = [slice(None)] * x.ndim

```

```

mask_shape = [1 if d != dim else -1 for d in range(x.ndim)]

for i in range(dim_size):
    slices[dim] = i
    mask = (idx == i).view(*mask_shape) if idx.ndim == 1 else (idx ==
    res[tuple(slices)] = torch.where(mask, x, 0).sum(dim=dim)

return res

# Test
x = torch.tensor([1,2,3,4], [3,2,4,6]))
idx = torch.tensor([0, 0, 1, 0])

y = scatter_add(x, idx, dim_size=2, dim=1)
print("Result:", y)
print("Expected:", torch.tensor([7, 3], [11, 4]))

```

```

Result: tensor([7, 3],
               [11, 4])
Expected: tensor([7, 3],
                 [11, 4])

```

Finally, we are ready to implement the graph convolution layer. We will use the approximate GCN by Kipf & Welling which we derived in the lecture.

Renormalized Laplacian:

$$\begin{aligned} & \mathbf{I} + \mathbf{D}^{-\frac{1}{2}} \mathbf{A} \mathbf{D}^{-\frac{1}{2}} \rightarrow \mathbf{\tilde{D}}^{-\frac{1}{2}} \\ & \mathbf{\tilde{A}} \mathbf{\tilde{D}}^{-\frac{1}{2}} \quad \text{with } \mathbf{\tilde{A}} = \mathbf{A} + \mathbf{I}_N, \quad \mathbf{\tilde{D}}_{ii} = \sum_j \mathbf{\tilde{A}}_{ij} \end{aligned}$$

Graph convolution:

$$\mathbf{g}_\theta * \mathbf{x} = \mathbf{\tilde{D}}^{-\frac{1}{2}} \mathbf{\tilde{A}} \mathbf{\tilde{D}}^{-\frac{1}{2}} \mathbf{x} \Theta$$

Since we use a sparse representation of the graph, you can not apply this directly. Instead, the formula has to be evaluated for the given edge indices. We will use the message passing paradigm to separate the graph convolution into message function

$$\mathbf{M}_t(\mathbf{x}_i^t, \mathbf{x}_j^t) = (\mathbf{\tilde{D}}^{-\frac{1}{2}} \mathbf{\tilde{A}} \mathbf{\tilde{D}}^{-\frac{1}{2}})_{ij} \mathbf{x}_j^t$$

aggregation

$$\mathbf{m}_i = \sum_{(i,j) \in E} \mathbf{M}_t(\mathbf{x}_i^t, \mathbf{x}_j^t)$$

and update function

$$\text{ReLU}(\mathbf{m}_i \Theta).$$

Task: Complete the GraphConv class below by implementing the three functions.

Hints:

- The edges for the tetris blocks, as extracted above, should already contain self-connections. Therefore, it is not necessary here to add them, which corresponds to adjusting the adjacency matrix with $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}_N$.
- Use the `scatter_add` function from above.

```
In [7]: class GraphConv(nn.Module):
    def __init__(self, n_in: int, n_out: int):
        super().__init__()
        self.n_in = n_in
        self.n_out = n_out
        self.theta = nn.Parameter(torch.empty((n_in, n_out)))
        torch.nn.init.xavier_uniform_(self.theta)

    def forward(self, x: torch.Tensor, edges: torch.Tensor) -> torch.Tensor:
        messages = self.compute_message(x, edges)
        aggregated_messages = self.aggregate(messages, edges, dim_size=x.shape[0])
        return self.compute_update(x, aggregated_messages)

    def compute_message(self, x: torch.Tensor, edges: torch.Tensor) -> torch.Tensor:
        src, dst = edges[:, 0], edges[:, 1]
        degrees = torch.bincount(src, minlength=x.shape[0]).float()
        norm = torch.where(degrees > 0, degrees ** -0.5, torch.zeros_like(degrees))
        return x[src] * (norm[src] * norm[dst]).unsqueeze(-1)

    def aggregate(self, messages: torch.Tensor, edges: torch.Tensor, dim_dst: int):
        dst = edges[:, 1].unsqueeze(-1).expand_as(messages)
        return scatter_add(messages, dst, dim=dim_dst, dim_size=dim_dst)

    def compute_update(self, x: torch.Tensor, m: torch.Tensor) -> torch.Tensor:
        return F.relu(x + m @ self.theta)
```

Here is an example network that uses the GraphConv, you have just implemented. Feel free to experiment with the number of conv layers, hidden features, output layers, etc.

```
In [8]: class GCN(nn.Module):
    def __init__(self, n_classes: int = 5, n_hidden: int = 20):
        super().__init__()
        self.conv1 = GraphConv(1, n_hidden)
        self.conv2 = GraphConv(n_hidden, n_hidden)
        self.conv3 = GraphConv(n_hidden, n_hidden)
        self.conv4 = GraphConv(n_hidden, n_hidden)
        self.outlayer1 = nn.Linear(n_hidden, n_hidden)
        self.outlayer2 = nn.Linear(n_hidden, n_classes)

    def forward(self, x: torch.Tensor, edges: torch.LongTensor, graph_idx: torch.Tensor):
```

```
# apply graph convs
x = self.conv1(x, edges)
x = self.conv2(x, edges)
x = self.conv3(x, edges)
x = self.conv4(x, edges)

# aggregate nodes by addition
num_graphs = torch.max(graph_idx) + 1
x = scatter_add(x, graph_idx, num_graphs)

# class output
x = F.relu(self.outlayer1(x))
x = self.outlayer2(x)
output = F.log_softmax(x, dim=1)
return output
```

Training

Here is some code to train your network. With the given network above, it should converge to the correct solution within about 15k steps.

```
In [9]: # initialize network
gcn = GCN()
```

```
In [10]: # initialize optimizer
optimizer = torch.optim.Adam(gcn.parameters(), lr=1e-3)
```

```
In [11]: #perform training
num_steps = 15000
batch = build_batch(tetris_edges)

x, edges, graph_idx, target = batch
for step in range(num_steps):
    optimizer.zero_grad()

    pred = gcn(x, edges, graph_idx)
    loss = F.nll_loss(pred, target)
    loss.backward()
    optimizer.step()

    if step % 1000 == 0:
        print(f"Step {step}: loss={loss}, prediction={pred.argmax(dim=1)}")
```

```

Step 0: loss=1.6132371425628662, prediction=tensor([2, 2, 2, 2, 2]), target=tensor([0, 1, 2, 3, 4])
Step 1000: loss=1.2123034000396729, prediction=tensor([2, 1, 2, 2, 2]), target=tensor([0, 1, 2, 3, 4])
Step 2000: loss=1.1158697605133057, prediction=tensor([2, 1, 2, 4, 4]), target=tensor([0, 1, 2, 3, 4])
Step 3000: loss=1.0859439373016357, prediction=tensor([2, 1, 2, 4, 4]), target=tensor([0, 1, 2, 3, 4])
Step 4000: loss=1.06459641456604, prediction=tensor([2, 1, 2, 4, 4]), target=tensor([0, 1, 2, 3, 4])
Step 5000: loss=1.046765923500061, prediction=tensor([2, 1, 2, 4, 4]), target=tensor([0, 1, 2, 3, 4])
Step 6000: loss=1.0301622152328491, prediction=tensor([2, 1, 2, 4, 4]), target=tensor([0, 1, 2, 3, 4])
Step 7000: loss=1.0141284465789795, prediction=tensor([2, 1, 2, 4, 4]), target=tensor([0, 1, 2, 3, 4])
Step 8000: loss=0.998451828956604, prediction=tensor([2, 1, 2, 4, 4]), target=tensor([0, 1, 2, 3, 4])
Step 9000: loss=0.9894272685050964, prediction=tensor([2, 1, 2, 3, 3]), target=tensor([0, 1, 2, 3, 4])
Step 10000: loss=0.9693816900253296, prediction=tensor([2, 1, 2, 4, 4]), target=tensor([0, 1, 2, 3, 4])
Step 11000: loss=0.9556148648262024, prediction=tensor([2, 1, 2, 4, 4]), target=tensor([0, 1, 2, 3, 4])
Step 12000: loss=0.944309413433075, prediction=tensor([2, 1, 2, 4, 4]), target=tensor([0, 1, 2, 3, 4])
Step 13000: loss=0.9299033880233765, prediction=tensor([2, 1, 2, 4, 4]), target=tensor([0, 1, 2, 3, 4])
Step 14000: loss=0.9186385869979858, prediction=tensor([0, 1, 2, 4, 4]), target=tensor([0, 1, 2, 3, 4])

```

Verify permutation invariance of edges

If everything has been done correctly, your network is invariant to the indexing of the tetris block nodes and edges, as long as the same (i.e. isomorph) graph is described. The following code creates permuted graphs, by shuffling the points of the tetris blocks. It should still give the same answer as above, no matter whether the network is trained or not.

```

In [12]: shuffled_points, shuffled_edges = create_dataset(shuffle=True)
         shuffled_batch = build_batch(shuffled_edges)
         x, edges, graph_idx, target = shuffled_batch

         output = gcn(x, edges, graph_idx)
         pred = output.argmax(dim=1)
         print(f"Prediction with shuffled nodes: {pred}")

```

Prediction with shuffled nodes: tensor([0, 1, 2, 4, 4])

In []: